



Python Loops - Complete Guide

From Beginner to Advanced with Real-Life Examples



What are Loops?

Loops let you **repeat code** without writing it again and again. Imagine telling a robot "walk 10 steps" — you do not say "step, step, step..." 10 times. You say "repeat stepping 10 times." That is a loop!

🧠 **Think of it like this:** Brushing your teeth — you do not brush just **ONE** tooth. You repeat the motion for **ALL** teeth. That is looping in real life!

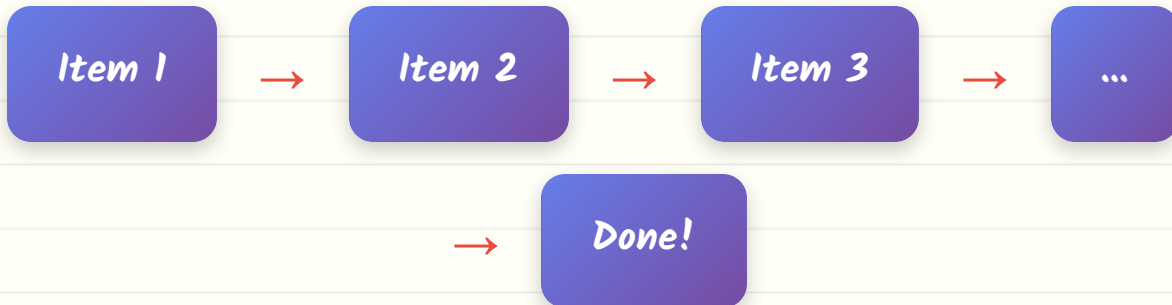


1. for Loops & Iteration

What is a for loop?

A **for** loop goes through a **sequence** (like a list, string, or range) and does something for **each item** one by one.

Syntax: `for item in sequence:`
`# do something with item`



Beginner Example 1: Print Numbers 1 to 5

```
# range(1, 6) gives: 1, 2, 3, 4, 5 for i in range(1, 6): print(i)
```

Output:

```
1
2
3
4
5
```

 **How it works:** `range(1, 6)` creates numbers from 1 up to (but NOT including) 6. The variable `i` becomes each number one at a time.

Beginner Example 2: Loop Through a List

```
fruits = ["Apple", "Banana", "Cherry"] for fruit in  
fruits: print(f"I love {fruit}! 🍎")
```

Output:

I love Apple! 🍎

I love Banana! 🍌

I love Cherry! 🍒

🌱 Beginner Example 3: Loop Through a String

```
name = "Python" for letter in name: print(letter)
```

Output:

P

y

t

h

o

n

🌿 Intermediate Example 1: Sum of Numbers

```
numbers = [10, 20, 30, 40, 50] total = 0 for num in  
numbers: total = total + num # or: total += num
```

```
print(f"Total sum = {total}")
```

Output:

Total sum = 150

 *Step by step: total starts at 0 → 0+10=10 → 10+20=30 → 30+30=60 → 60+40=100 → 100+50=150*

Intermediate Example 2: `enumerate()` - Get Index + Value

```
colors = ["Red", "Green", "Blue"] for index, color  
in enumerate(colors): print(f"{index}: {color}")
```

Output:

0: Red

1: Green

2: Blue

 *`enumerate()` gives you BOTH the position (index) and the value.
Super useful!*

Intermediate Example 3: `zip()` - Loop Two Lists Together

```
names = ["Alice", "Bob", "Charlie"] scores = [85,  
92, 78] for name, score in zip(names, scores):  
print(f"{name} scored {score}")
```

Output:

Alice scored 85

Bob scored 92

Charlie scored 78

🔥 Advanced Example 1: List Comprehension (Loop in One Line!)

```
# Traditional way (3 lines) squares = [] for x in range(1, 6): squares.append(x ** 2) # List Comprehension way (1 line!) squares = [x ** 2 for x in range(1, 6)] print(squares)
```

Output:

[1, 4, 9, 16, 25]

🔥 Advanced Example 2: Dictionary Comprehension

```
words = ["cat", "elephant", "dog"] # Create dict: word -> length lengths = {word: len(word) for word in words} print(lengths)
```

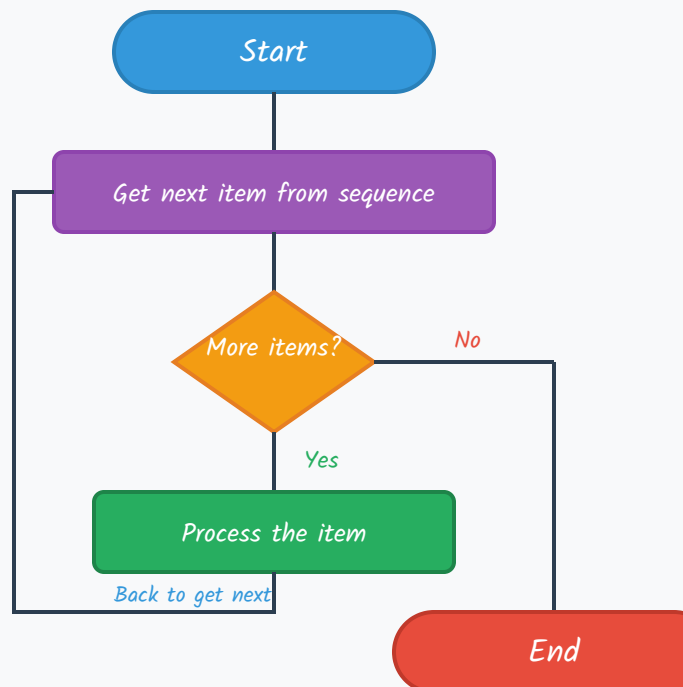
Output:

```
{'cat': 3, 'elephant': 8, 'dog': 3}
```

Real Life Analogy



Packing a School Bag: You have a list of books. You pick up Book 1, put it in bag. Pick up Book 2, put it in bag. Repeat until all books are packed. That is a **for** loop — you know exactly how many items there are, and you process each one!



Flowchart: for loop



2. while Loops

What is a while loop?

A `while` loop keeps running *as long as a condition is true*. You do not need to know how many times — it just keeps going until something changes!

Syntax: `while condition:`

`# do something`

**# IMPORTANT: change something so
condition can become false!**



Infinite Loop Danger! If you forget to update the variable inside a while loop, it will run FOREVER and crash your program. Always make sure the condition can eventually become False!

🌱 Beginner Example 1: Count from 1 to 5

```
count = 1 while count <= 5: print(count) count =  
count + 1 # Do not forget this! print("Done! 🎉")
```

Output:

```
1  
2  
3  
4
```

5

Done! 🎉

🔍 *Watch carefully:* count starts at 1. Each loop, we add 1. When count becomes 6, the condition `6 <= 5` is False, so the loop stops.

🌱 Beginner Example 2: User Input Validation

```
password = "python123" guess = "" while guess != password: guess = input("Enter password: ") if guess != password: print("Wrong! Try again. ✖") print("Access granted! ✔")
```

Output:

```
Enter password: hello
Wrong! Try again. ✖
Enter password: python123
Access granted! ✔
```

🌿 Intermediate Example 1: Guessing Game with Limited Tries

```
secret = 42 guess = 0 attempts = 0 max_attempts = 5
while guess != secret and attempts < max_attempts:
    guess = int(input("Guess the number (1-100): "))
    attempts += 1 if guess < secret: print("Too low!
```

```

    ") elif guess > secret: print("Too high! 📈") if
guess == secret: print(f"🎉 Correct! You got it in
{attempts} tries!") else: print(f"😞 Out of tries!
The number was {secret}.")

```

🌿 Intermediate Example 2: Factorial Using while

```

n = 5 factorial = 1 i = 1 while i <= n: factorial *=
i # same as: factorial = factorial * i i += 1
print(f"{n}! = {factorial}")

```

Output:

5! = 120

📊 Step by step: $1 \times 1 = 1 \rightarrow 1 \times 2 = 2 \rightarrow 2 \times 3 = 6 \rightarrow 6 \times 4 = 24 \rightarrow 24 \times 5 = 120$

🔥 Advanced Example: Fibonacci Sequence

```

# Generate first 10 Fibonacci numbers # 0, 1, 1, 2,
3, 5, 8, 13, 21, 34... a, b = 0, 1 count = 0 n = 10
while count < n: print(a, end=" ") temp = a + b a =
b b = temp count += 1

```

Output:

0 1 1 2 3 5 8 13 21 34

Real Life Analogy



Washing Dishes: "While there are dirty dishes in the sink, keep washing." You do not know exactly how many dishes there are — you just keep going until the sink is empty. That is a **while** loop!

<i>for loop</i>	<i>while loop</i>
You KNOW how many times	You DO NOT know how many times
Best for: lists, strings, ranges	Best for: user input, waiting for events
Automatically stops at end	You must update the condition
<pre>for i in range(5):</pre>	<pre>while i < 5:</pre>

3. break Statement

What does break do?

break is like an **emergency exit** — it immediately stops the loop and jumps out, no matter what!



break = "STOP EVERYTHING AND GET OUT OF THE LOOP RIGHT NOW!"

Beginner Example: Find First Even Number

```
numbers = [1, 3, 5, 8, 9, 10] for num in numbers: if  
num % 2 == 0: print(f"First even number: {num}")  
break # Stop! We found it, no need to check rest
```

Output:

First even number: 8

 **Why break?** Without break, the loop would keep checking 9 and 10 even though we already found our answer. break saves time!

Intermediate Example: Search in a List

```
students = ["Alice", "Bob", "Charlie", "David"]  
target = "Charlie" found = False for i, student in  
enumerate(students): if student == target:  
print(f"Found {target} at position {i}! ) found =  
True break if not found: print(f"{target} not found.  
)
```

Output:

Found Charlie at position 2! 

Advanced Example: Prime Number Check

```
num = 29 is_prime = True for i in range(2, int(num
** 0.5) + 1): if num % i == 0: is_prime = False
print(f"{num} is divisible by {i}") break # No need
to check further! if is_prime: print(f"{num} is a
PRIME number! ★")
```

Output:

29 is a PRIME number! ★

🧠 **Smart optimization:** We only check up to square root of num. If no divisor found by then, it is prime. break stops as soon as we find ONE divisor!

🌍 Real Life Analogy



Fire Alarm in a Building: You are checking every room for a fire. As soon as you find ONE room with fire, you pull the alarm (break) and evacuate everyone. You do not keep checking other rooms — the job is done!



4. continue Statement

What does continue do?

continue says: "Skip the rest of THIS round, and go to the NEXT round immediately!" It does not stop the loop — it just skips the current item.



continue = "Skip this one, next please!"



Beginner Example: Skip Even Numbers

```
for i in range(1, 11): if i % 2 == 0: continue #  
Skip even numbers print(i)
```

Output:

1
3
5
7
9

What happened? When $i=2$, the `if` is `True`, so `continue` runs → skips `print(i)`. Then $i=3$, `if` is `False`, so `print` runs. And so on!



Intermediate Example: Process Only Valid Emails

```
emails = ["alice@gmail.com", "", "bob@yahoo.com",  
"invalid", "charlie@outlook.com"] for email in  
emails: if "@" not in email: print(f"Skipping  
invalid: {email}") continue print(f"✓ Sending email  
to: {email}")
```

Output:

```
✓ Sending email to: alice@gmail.com
Skipping invalid:
✓ Sending email to: bob@yahoo.com
Skipping invalid: invalid
✓ Sending email to: charlie@outlook.com
```

Advanced Example: Process Data with Multiple Filters

```
data = [10, -5, 20, 0, -3, 15, 8] total = 0 count = 0
for value in data:
    if value <= 0:
        print(f"Skipping non-positive: {value}")
        continue
    if value % 2 != 0:
        print(f"Skipping odd: {value}")
        continue
    total += value
    count += 1
    print(f"Added: {value}")
print(f"Sum of valid even positives: {total}")
print(f"Count: {count}")
```

Output:

```
Added: 10
Skipping non-positive: -5
Added: 20
Skipping non-positive: 0
Skipping non-positive: -3
Skipping odd: 15
Added: 8
```

```
Sum of valid even positives: 38
```

```
Count: 3
```

Real Life Analogy



Inspecting Apples: You are checking apples in a basket. If one is rotten, you say "Skip this one" (*continue*) and pick the next apple. You do not stop checking — you just ignore the bad ones and keep going!



break vs continue — Do not Confuse!



break = STOP the entire loop (emergency exit)



continue = SKIP this round, go to next round

Think of a race: *break* = quit the race entirely. *continue* = skip one lap, keep racing.



5. else Clause in Loops

What is it?

Yes, loops can have an **else** block! It runs **ONLY** if the loop completed **normally** (did NOT encounter a **break**). If *break* happened, *else* is skipped.

Syntax: `for item in sequence:`

`# do stuff`

`if condition:`

`break`

`else:`

`# runs ONLY if loop finished without
break!`

Beginner Example: Search with "Not Found" Message

```
fruits = ["apple", "banana", "cherry"] target =  
"mango" for fruit in fruits: if fruit == target:  
print(f"Found {target}! 🎉") break else: print(f"  
{target} not found in the list. 😞")
```

Output:

mango not found in the list. 😞

 *Why this is cool:* The else runs because the loop finished all items WITHOUT hitting break. No need for a separate `found` flag variable!

Intermediate Example: Prime Number Check (Cleaner Version)

```
num = 17 for i in range(2, num): if num % i == 0:
print(f"{num} is NOT prime (divisible by {i})")
break else: print(f"{num} is PRIME! ★")
```

Output:

17 is PRIME! ★

✨ *Elegant! The else belongs to the FOR loop, not the if. It runs when the for completes normally — meaning no divisor was found!*

🔥 Advanced Example: Password Validator with while-else

```
attempts = 0 max_attempts = 3 password = "secret"
while attempts < max_attempts: guess =
input("Password: ") attempts += 1 if guess ==
password: print("✅ Access granted!") break else:
print("❌ Account locked! Too many failed
attempts.")
```

Output (if all 3 wrong):

Password: wrong1

Password: wrong2

Password: wrong3

❌ Account locked! Too many failed attempts.



Real Life Analogy



Searching for Keys: You search every pocket. If you find them (break), you are happy and leave. If you check ALL pockets and do not find them (loop finishes), then you say "I could not find my keys anywhere!" — that is the else clause!

Situation	Does else run?
Loop finishes all iterations	✓ YES
Loop hits <code>break</code>	✗ NO
Loop hits <code>continue</code>	✓ YES (continue does not stop the loop)
Empty sequence (nothing to loop)	✓ YES (loop "finished" immediately)



6. Nested Loops

What are Nested Loops?

A loop **inside another loop**. The inner loop runs COMPLETELY for each single iteration of the outer loop. Like hours and minutes on a clock!

Syntax: `for outer in range(3):` ← *Outer loop*
 `for inner in range(3):` ← *Inner loop*
 `# do something`

Beginner Example 1: Simple Multiplication Table

```
for i in range(1, 4): for j in range(1, 4): print(f"
{i} x {j} = {i*j}") print("---") # Separator after
each row
```

Output:

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
---
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
---
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
---
```

 **How it works:** Outer $i=1 \rightarrow$ inner j runs 1,2,3. Then outer $i=2 \rightarrow$ inner j runs 1,2,3 again. Then $i=3$. Total: $3 \times 3 = 9$ iterations!

Beginner Example 2: Print a Right Triangle

```
rows = 5 for i in range(1, rows + 1): for j in
range(i): print("*", end=" ") print()
```

Output:

```
*
* *
* * *
* * * *
* * * * *
```

Intermediate Example: Find Common Elements

```
list_a = [1, 2, 3, 4, 5] list_b = [4, 5, 6, 7, 8]
common = [] for a in list_a: for b in list_b: if a
== b: common.append(a) break # Found match, no need
to check more print(f"Common elements: {common}")
```

Output:

```
Common elements: [4, 5]
```

Intermediate Example: Matrix Transpose

```
# Original matrix (2D list) matrix = [ [1, 2, 3],  
[4, 5, 6], [7, 8, 9] ] # Transpose: rows become  
columns transpose = [] for i in  
range(len(matrix[0])): row = [] for j in  
range(len(matrix)): row.append(matrix[j][i])  
transpose.append(row) for row in transpose:  
print(row)
```

Output:

```
[1, 4, 7]  
[2, 5, 8]  
[3, 6, 9]
```

Advanced Example: Bubble Sort Algorithm

```
numbers = [64, 34, 25, 12, 22, 11, 90] n =  
len(numbers) for i in range(n): for j in range(0, n  
- i - 1): if numbers[j] > numbers[j + 1]: # Swap!  
numbers[j], numbers[j + 1] = numbers[j + 1],  
numbers[j] print(f"Sorted: {numbers}")
```

Output:

```
Sorted: [11, 12, 22, 25, 34, 64, 90]
```

 **How Bubble Sort works:** Outer loop controls passes. Inner loop compares adjacent pairs and swaps if out of order. After each pass, the largest unsorted number "bubbles" to the end!

Advanced Example: Pattern with break in Nested Loop

```
# Find first pair that sums to target
nums = [2, 7, 11, 15]
target = 9
found = False
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        if nums[i] + nums[j] == target:
            print(f"Pair found: ({nums[i]}, {nums[j]}) at indices ({i}, {j})")
            found = True
            break
    if found:
        break
```

Output:

Pair found: (2, 7) at indices (0, 1)

 **Two breaks needed!** The inner break exits the inner loop. But we are still in the outer loop! So we need a flag + second break to exit the outer loop too.

Real Life Analogy



Movie Theater Seats: Outer loop = "Go to Row 1, then Row 2, then Row 3..." Inner loop = "Check Seat A, then Seat B, then Seat C..." For EACH row, you check ALL seats. That is nested loops!



Performance Warning: Nested loops can be SLOW! If outer loop runs N times and inner loop runs N times, total operations = $N \times N = N$ squared. For $N=1000$, that is 1 MILLION operations! Be careful with large data.



7. Loop Optimization Techniques

Why Optimize?

Bad loops can make your program SLOW. A few smart changes can make it 10x or even 100x faster! Here are the techniques every Python programmer should know:



Technique 1: Use Built-in Functions (They are Written in C!)

```
# ❌ SLOW: Manual loop numbers = [1, 2, 3, 4, 5]
total = 0 for num in numbers: total += num # ✅
FAST: Built-in sum() total = sum(numbers) # Other
built-ins to use: # max(), min(), len(), any(),
all(), sorted()
```

Both give: 15

But `sum()` is 5-10x faster because it is written in C!

🌱 Technique 2: Use List Comprehensions

```
numbers = [1, 2, 3, 4, 5] # ❌ SLOW: Traditional for loop
squares = []
for n in numbers:
    squares.append(n ** 2)
# ✅ FAST: List comprehension
squares = [n ** 2 for n in numbers]
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

⚡ **Speed boost:** List comprehensions are 2-3x faster and use less memory!

🌱 Technique 3: Use Generators for Large Data

```
# ❌ BAD: Creates a HUGE list in memory
big_list = [x ** 2 for x in range(1000000)]
total = sum(big_list)
# ✅ GOOD: Generator – calculates one at a time, saves memory!
big_gen = (x ** 2 for x in range(1000000))
total = sum(big_gen)
print(f"Sum = {total}")
```

Output:

Sum = 333332833333500000

Generator uses almost ZERO extra memory!

🧠 *How generators work:* Instead of storing all 1 million squares in RAM, it calculates each square only when needed, then forgets it. Perfect for huge datasets!

🌿 Technique 4: Avoid Repeated Calculations in Loops

```
data = [1, 2, 3, 4, 5] # ❌ BAD: Calculates len(data) EVERY iteration
for i in range(len(data)):
    print(data[i]) # ✅ GOOD: Calculate once, store in variable
n = len(data)
for i in range(n):
    print(data[i]) # ✅ EVEN BETTER: Direct iteration (no indexing needed)
for item in data:
    print(item)
```

🔥 Technique 5: Use Sets for Membership Testing

```
# ❌ SLOW: Checking if item in list - O(N) each time
big_list = list(range(100000))
for i in range(1000):
    if i in big_list: # Checks 100k items each time!
        print(i) # ✅ FAST: Convert to set first - O(1) lookup!
big_set = set(big_list)
for i in
```

```
range(1000): if i in big_set: # Instant lookup!  
print(i)
```

Set lookup: $O(1)$ – instant!

List lookup: $O(N)$ – scans entire list

For 1000 checks: Set = ~1000 ops, List = ~100

MILLION ops!

🔥 Technique 6: Early Exit with break

```
numbers = [1, 3, 5, 8, 10, 12] # ❌ BAD: Checks ALL  
items even after finding answer for num in numbers:  
if num % 2 == 0: print(f"First even: {num}") # ✅  
GOOD: Exit immediately after finding answer for num  
in numbers: if num % 2 == 0: print(f"First even:  
{num}") break # Stop! We are done.
```



🚀 Pro Tip: The "Pythonic" Way

Python has a philosophy: "There should be one — and preferably only one — obvious way to do it."

When in doubt, prefer:

- ✅ Built-in functions over manual loops
- ✅ List comprehensions over for-loops for simple cases

-  Direct iteration (`for item in list`) over indexing (`for i in range(len(list))`)
-  Generators over lists for huge data
-  Sets over lists for membership tests

Technique	When to Use	Speed Gain
Built-in functions	<code>sum</code> , <code>max</code> , <code>min</code> , <code>len</code> , etc.	5-10x
List comprehensions	Creating new lists from old ones	2-3x
Generators	Large datasets, one-pass operations	Infinite memory savings!
Sets for lookup	Frequent "in" checks	1000x+ for large data
<code>break</code> early	Search operations	Up to Nx (skips rest)

8. Practice Problems (Beginner → Advanced)



Problem 1: Print 1 to 10

Beginner

Use a for loop to print numbers 1 through 10.

[Show Solution](#)**Problem 2: Countdown Timer****Beginner**

Use a while loop to count down from 10 to 1, then print "BLAST OFF! 🚀"

[Show Solution](#)**Problem 3: Sum of Even Numbers****Beginner**

Calculate the sum of all even numbers from 1 to 20 using a for loop.

[Show Solution](#)**Problem 4: Multiplication Table****Beginner**

Print the multiplication table of 5 (5x1 to 5x10).

[Show Solution](#)**Problem 5: Find Factorial****Intermediate**

Write a program to calculate factorial of a number using a while loop.

Factorial of 5 = $5 \times 4 \times 3 \times 2 \times 1 = 120$

[Show Solution](#)**Problem 6: Reverse a String****Intermediate**

Reverse a string without using the `[::-1]` shortcut. Use a loop!

[Show Solution](#)**Problem 7: Find First Negative Number****Intermediate**

Given a list of numbers, find and print the *FIRST* negative number. Use `break`!

[Show Solution](#)**Problem 8: Skip Multiples of 3****Intermediate**

Print numbers 1 to 20, but skip multiples of 3 using `continue`.

[Show Solution](#)**Problem 9: Prime Number Check with else****Intermediate**

Check if a number is prime using a `for-else` loop.

[Show Solution](#)**Problem 10: Print a Pyramid Pattern****Intermediate**

Use nested loops to print this pyramid (height = 5):

```
  *
 * * *
* * * * *
* * * * * *
* * * * * * *
```

[Show Solution](#)**Problem 11: Two Sum Problem****Advanced**

Find two numbers in a list that add up to a target. Return their indices.

Use nested loops with break!

[Show Solution](#)**Problem 12: Optimized Prime Sieve****Advanced**

Find all prime numbers up to 50 using the Sieve of Eratosthenes algorithm.

[Show Solution](#)

Problem 13: Password Strength Checker

Advanced

Check if a password is strong. Strong = at least 8 chars, has uppercase, lowercase, digit, and special character.

[Show Solution](#)

Problem 14: Flatten a 2D List

Advanced

Convert a nested list into a single flat list using nested loops.

[Show Solution](#)

Challenge: FizzBuzz

Classic!

Print numbers 1 to 100. But for multiples of 3 print "Fizz", for multiples of 5 print "Buzz", for multiples of both print "FizzBuzz".

[Show Solution](#)

9. Summary: Which Loop to Use?

- 1 *Use **for loop** when you know how many times to repeat (lists, strings, ranges)*
- 2 *Use **while loop** when you don't know — keep going until a condition changes*
- 3 *Use **break** when you found what you need — stop searching!*
- 4 *Use **continue** when you want to skip bad data but keep processing*
- 5 *Use **loop-else** when you need a "not found" message after searching*
- 6 *Use **nested loops** for grids, matrices, pairs, and combinations*
- 7 *Optimize with built-ins, comprehensions, generators, and sets!*

 Made with  for Learning | Python Loops Complete Guide

Remember: The best loop is the one that makes your code clean, fast, and easy to understand!